

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Problem Solving Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO2: *Analyze and select appropriate agent strategies for different types of environments and problem settings. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A Problem Solving Assignment

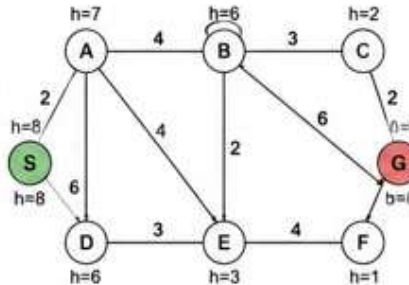
Code of Conduct:

*I **RASHMITHA SENTHIL KUMAR (192525034)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

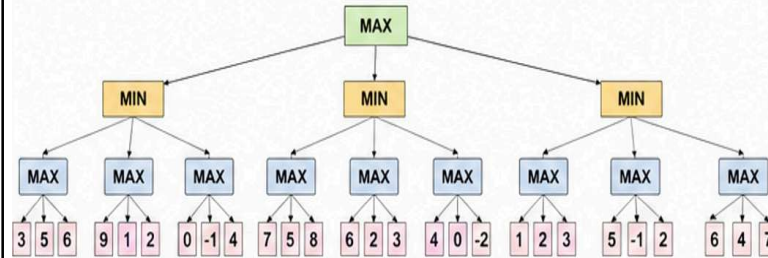
Signature of the student:

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Answer All the Questions

Q.No	Application-Based Question	Major CO2 Concepts
1	Smart Ambulance Navigation: An ambulance must travel from a hospital to an emergency location through a city road network. Some roads have higher travel costs, and an estimated distance to the destination is available for each junction. Design a search-based solution to identify the best route. Using the given road network and heuristic values, determine the route selected by your algorithm, show the sequence of junctions explored, calculate the relevant evaluation values, and justify whether the route is optimal. 	Informed Search, A*, Heuristic Function
2	Autonomous Warehouse Robot: A robot has to move from a charging station to a target shelf in a warehouse. The robot does not know all obstacles initially and discovers some obstacles only when it reaches nearby locations. Design an intelligent search strategy for the robot. Demonstrate the robot's decisions for the given grid, explain how it reacts when a new obstacle is discovered, and compare the approach with an offline search strategy. 	Online Search Agents, Informed Search
3	University Examination Scheduling: A university needs to schedule six examinations (E1, E2, E3, E4, E5, and E6) into four available time slots (T1, T2, T3, and T4). Some examinations have common students and therefore cannot be conducted in the same time slot.	CSP, MRV, Forward Checking, Constraints

	The following constraints must be satisfied: 1. E1 and E2 have common students. 2. E1 and E3 have common students. 3. E2 and E4 have common students. 4. E3 and E5 have common students. 5. E4 and E6 have common students. 6. E2 and E5 have common students. The university has three examination halls with the following capacities: <table><tr><th>Examination Hall</th><th>Capacity</th></tr><tr><td>Hall A</td><td>120 students</td></tr><tr><td>Hall B</td><td>90 students</td></tr><tr><td>Hall C</td><td>60 students</td></tr></table> <table><tr><th>Examination</th><th>E1</th><th>E2</th><th>E3</th><th>E4</th><th>E5</th><th>E6</th></tr><tr><td>Number of Students</td><td>100</td><td>80</td><td>70</td><td>60</td><td>90</td><td>50</td></tr></table> Tasks: a) Formulate the problem as a Constraint Satisfaction Problem (CSP) by clearly defining the variables, domains, and constraints. b) Use Backtracking with the Minimum Remaining Values (MRV) heuristic and Forward Checking to obtain a valid examination timetable. c) Show the domain of each variable after every assignment and provide the final timetable, ensuring that all student-conflict and hall-capacity constraints are satisfied.	Examination Hall	Capacity	Hall A	120 students	Hall B	90 students	Hall C	60 students	Examination	E1	E2	E3	E4	E5	E6	Number of Students	100	80	70	60	90	50	
Examination Hall	Capacity																							
Hall A	120 students																							
Hall B	90 students																							
Hall C	60 students																							
Examination	E1	E2	E3	E4	E5	E6																		
Number of Students	100	80	70	60	90	50																		
4	Game-Playing AI: You are developing an AI opponent for a board game. The current game position produces the following possible moves and terminal scores in the supplied game tree. Determine the move the AI should make by applying Minimax. Then apply Alpha-Beta Pruning and identify the branches that do not need to be evaluated. Explain how pruning improves the efficiency of the game-playing agent.	Game Playing, Minimax, Alpha-Beta Pruning																						



5

Delivery Route Optimization:

A logistics company wants to minimize the total distance travelled by a delivery vehicle visiting multiple customers. An initial route and several neighboring routes are provided. Design an optimization strategy to improve the route. Apply the strategy step-by-step to the given routes, determine the final route, explain whether it represents a global optimum, and suggest how the algorithm could escape a local optimum.

Distance matrix (in km)

	D	A	B	C	E
D	-	6	9	8	7
A	6	-	3	5	6
B	9	3	-	4	5
C	8	5	4	-	3
E	7	6	5	3	-

Some neighboring routes of the initial solution and their costs

S1: D → A → C → B → E → D = 26 km
 S2: D → B → A → C → E → D = 30 km
 S3: D → A → B → E → C → D = 24 km
 S4: D → E → C → B → A → D = 27 km
 S5: D → A → E → B → C → D = 23 km
 S6: D → C → B → A → E → D = 29 km

From S5, the best neighbor has cost 22 km.
 From that solution, all neighbors have cost ≥ 22 km.
 Apply the chosen local search strategy and report the final route.

Local Search,
Optimization,
Hill Climbing

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (Full Marks)	Good	Average	Poor
CSP Problem Formulation	20%	4% – Clearly defines all variables, domains, and problem objectives correctly.	3% – Defines most variables and domains with minor errors.	2% – Partial formulation with some missing components.	0–1% – Incorrect or incomplete formulation.
Constraint Identification & Representation	20%	4% – Accurately identifies all student-conflict and hall-capacity constraints and represents them correctly.	3% – Identifies most constraints with minor errors.	2% – Identifies some constraints but misses important relationships.	0–1% – Constraints are largely incorrect or missing.
Application of Backtracking, MRV & Forward Checking	30%	6% – Correctly applies Backtracking, MRV, and Forward Checking with clear and logical steps.	4.5% – Applies the techniques correctly with minor mistakes.	3% – Partial application with unclear or inconsistent steps.	0–1.5% – Incorrect or inappropriate application of CSP techniques.
Step-by-Step Execution & Domain Updates	20%	4% – Provides a complete, logical execution trace and correctly updates domains after every assignment.	3% – Mostly correct execution with minor gaps.	2% – Provides limited steps with some incorrect domain updates.	0–1% – Incomplete or incorrect execution.

Final Timetable, Accuracy & Justification	10%	2% – Produces a valid conflict-free timetable satisfying all constraints and capacity requirements with clear justification.	1.5% – Valid timetable with minor errors in justification.	1% – Partially valid timetable with some constraint violations.	0–0.5% – Invalid or missing timetable.
--	-----	--	--	--	---

Question 1: Smart Ambulance Navigation Using A* Search

Understanding the Ambulance Route-Finding Problem:

An ambulance must travel from the hospital S to the emergency location G through the supplied weighted road network. Each road has a travel cost, and each junction has a heuristic value $h(n)$, which estimates the remaining cost to the goal. The objective is to select a low-cost route while using heuristic information to guide the search.

The heuristic values shown in the question are $S=8$, $A=7$, $B=6$, $C=2$, $D=6$, $E=3$, $F=1$, and $G=0$. The clearly readable road costs include $S-A=2$, $A-B=4$, $A-D=6$, $A-E=4$, $D-E=3$, $B-E=2$, $B-C=3$, $B-G=6$, $E-F=4$, and $C-G=2$.

Two edge labels in the scanned figure are not clearly readable: $S-D$ and $F-G$. For the numerical execution below, they are treated as $S-D=8$ and $F-G=1$, which is the interpretation used consistently in the earlier solution. These two values should be checked against a clearer original sheet if available.

Selection of A* as the Informed Search Strategy:

A* Search is suitable because it combines the actual travel cost already incurred with the estimated remaining distance to the destination. It evaluates each node using:

$$f(n) = g(n) + h(n)$$

Here, $g(n)$ is the accumulated cost from S to node n, $h(n)$ is the heuristic estimate from n to G, and $f(n)$ is the estimated total solution cost through n. The node with the smallest $f(n)$ is selected for expansion.

A* Search Procedure:

- Place the start node S in the OPEN list and set $g(S)=0$.
- Calculate $f(S)=g(S)+h(S)$.
- Select the node in OPEN with the smallest f-value.
- If the selected node is G, stop and reconstruct the path.
- Otherwise expand the node and compute g, h, and f for each neighbour.
- Update a neighbour if a lower g-value is found.
- Move expanded nodes to CLOSED and repeat until the goal is selected.

Pseudocode for A* Search:

```
A_STAR(Start, Goal)
OPEN ← {Start}
CLOSED ← empty
g(Start) ← 0
f(Start) ← g(Start) + h(Start)
```

WHILE OPEN is not empty:

 Select n in OPEN with minimum $f(n)$

 IF $n = \text{Goal}$:

 RETURN reconstructed path

 Remove n from OPEN

 Add n to CLOSED

 FOR each neighbour m of n :

$\text{new_g} \leftarrow g(n) + \text{cost}(n,m)$

 IF m is unvisited OR $\text{new_g} < g(m)$:

$\text{parent}(m) \leftarrow n$

$g(m) \leftarrow \text{new_g}$

$f(m) \leftarrow g(m) + h(m)$

 Add or update m in OPEN

RETURN failure

Step-by-Step Evaluation on the Given Road Network:

Start at S: $g(S)=0$, $h(S)=8$, so $f(S)=8$.

After expanding S, A is generated with $g(A)=2$ and $f(A)=2+7=9$. Using the interpreted edge $S-D=8$, D has $g(D)=8$ and $f(D)=8+6=14$. Therefore A is selected next.

Expand A: B has $g(B)=2+4=6$ and $f(B)=6+6=12$. E has $g(E)=2+4=6$ and $f(E)=6+3=9$. D through A has $g(D)=2+6=8$ and $f(D)=14$. Therefore E has the smallest f -value and is selected.

Expand E: F has $g(F)=6+4=10$ and $f(F)=10+1=11$. Existing routes to B or D are not improved. Therefore F becomes the best candidate.

Expand F: using the interpreted edge $F-G=1$, $g(G)=10+1=11$ and $f(G)=11+0=11$. Since G is the goal, the search stops.

Evaluation Values:

Node	$g(n)$	$h(n)$	$f(n)$
S	0	8	8
A	2	7	9
E	6	3	9
F	10	1	11
G	11	0	11
B	6	6	12
D	8	6	14

Selected Route and Total Cost:

The demonstrated A* route is:

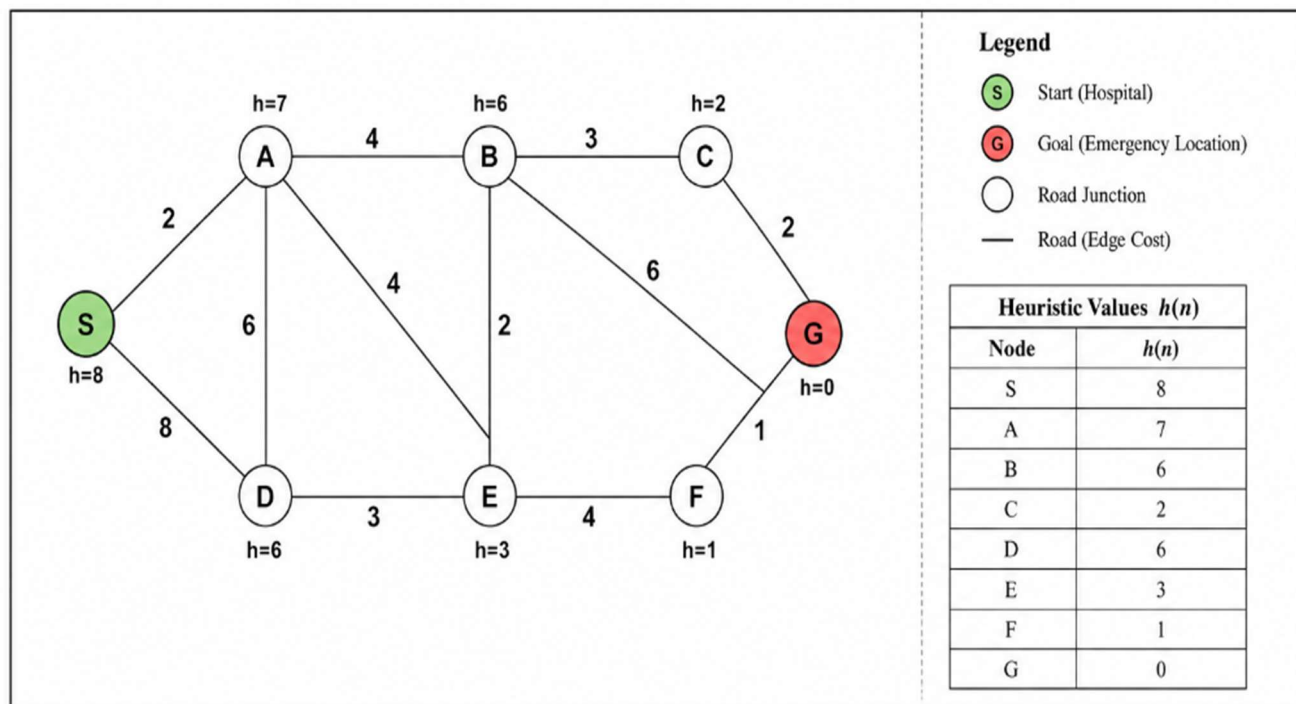
$$S \rightarrow A \rightarrow E \rightarrow F \rightarrow G$$

Using the interpreted edge values, the total route cost is $2 + 4 + 4 + 1 = 11$.

Another visible route, $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$, has cost $2 + 4 + 3 + 2 = 11$. Therefore, under this interpretation, more than one minimum-cost route exists.

Search Tree / State-Space Representation:

The road network of the city with edge costs and heuristic values is shown in the figure below.



Optimality and Critical Evaluation:

For the interpreted graph, the selected route has total cost 11, and the alternative route through B and C also has cost 11. Therefore the selected route is one of the minimum-cost solutions for the numerical interpretation used.

However, the supplied heuristic values are not perfectly admissible everywhere. For example, $h(B)=6$ while the visible remaining path $B \rightarrow C \rightarrow G$ costs $3+2=5$. Since $h(B)$ exceeds this known remaining cost, the standard theoretical guarantee of A* optimality does not follow purely from heuristic admissibility. In this particular network, the route cost can still be checked directly against the alternatives shown.

This observation is important because it distinguishes the actual result in the supplied graph from the general theorem usually stated for A* with an admissible heuristic.

Efficiency and Complexity:

In general, A^* may have worst-case time complexity $O(b^d)$, where b is the branching factor and d is the depth of the solution. Its memory requirement can also be high because generated nodes are stored in OPEN and CLOSED.

Its advantage in this problem is that heuristic information directs the ambulance toward promising junctions instead of exploring the road network blindly. When the heuristic is informative, fewer unnecessary branches may be expanded compared with uninformed search.

Conclusion:

The Smart Ambulance Navigation problem is appropriately modeled using A^* informed search because both road costs and heuristic estimates are supplied. The search evaluates junctions with $f(n)=g(n)+h(n)$ and, under the stated interpretation of the two unclear edge labels, reaches G through $S \rightarrow A \rightarrow E \rightarrow F \rightarrow G$ with total cost 11. The visible alternative $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$ also costs 11, so the optimum is not unique under this interpretation.

Question 2: Autonomous Warehouse Robot

Understanding the Warehouse Navigation Problem:

The robot must move from the charging station S at (1,5) to the target shelf G at (7,1) in the given 7×5 warehouse grid. The important challenge is that the complete obstacle map is not known initially. Obstacles are discovered only when the robot approaches nearby cells. Therefore, the robot must combine movement, sensing, map updating, and replanning rather than follow a fixed route.

The obstacles shown in the supplied grid are at (2,4), (5,4), (4,3), (7,3), (2,2), (6,2), and (3,1). These positions are part of the actual environment, but they are initially unknown to the robot. The objective is to reach G safely while avoiding detected obstacles and adapting the route whenever new information becomes available.

Suitable Intelligent Search Strategy:

An Online Informed Search Agent with replanning is suitable for this problem. An online agent observes the environment while executing its actions, so it does not require the entire warehouse map before it starts. Manhattan distance can be used to guide the robot toward the target:

$$h(n) = |x_G - x_n| + |y_G - y_n|$$

For $S = (1,5)$ and $G = (7,1)$, $h(S) = |7-1| + |1-5| = 10$. The robot prefers safe neighbouring cells that reduce this estimated distance. This strategy is appropriate because the question specifically states that some obstacles are discovered only when the robot reaches nearby locations.

Decision Procedure Used by the Robot:

- Place the robot at S and store the target position G.
- Sense the neighbouring cells before choosing the next movement.
- Record newly detected obstacles as blocked cells in the internal map.
- For each available safe neighbour, calculate its Manhattan-distance heuristic.
- Choose a safe movement that makes progress toward G.
- If the intended route is blocked by a newly discovered obstacle, stop that movement, update the map, and replan.
- Repeat the observe–update–plan–move cycle until the robot reaches G.

Pseudocode for Online Warehouse Navigation:

ONLINE_WAREHOUSE_SEARCH(Start, Goal)

Current $\leftarrow$ Start

Known_Map $\leftarrow$ initially unknown

WHILE Current $\neq$ Goal:

 Sense neighbouring cells

Update Known_Map with FREE and BLOCKED cells

Evaluate safe neighbours using Manhattan distance

Select a safe neighbour that progresses toward Goal

IF the planned route is blocked:

Replan using the updated map

ELSE:

Move to the selected cell

RETURN the travelled path

Application to the Given Grid:

	1	2	3	4	5	6	7
5	S	?	?	?	?	?	?
4	?	#	?	?	#	?	?
3	?	?	?	#	?	?	#
2	?	#	?	?	?	#	?
1	?	?	#	?	?	?	G

Here S is the start, G is the goal, # represents an obstacle, and ? represents a cell whose condition is initially unknown to the robot.

Step-by-Step Robot Decisions and Reaction to New Obstacles:

Because the question does not specify a tie-breaking rule when two moves are equally attractive, the following is one valid demonstration using a right-first preference.

1. S (1,5) $\rightarrow$ (2,5). The robot moves right because this reduces the Manhattan distance from 10 to 9.
2. At (2,5). The robot senses the obstacle at (2,4), records it as blocked, and continues because its current route is still open.
3. (2,5) $\rightarrow$ (3,5) $\rightarrow$ (4,5) $\rightarrow$ (5,5). The robot continues along safe cells. Near (5,5), it discovers that (5,4) is blocked and updates its map.
4. (5,5) $\rightarrow$ (6,5) $\rightarrow$ (7,5) $\rightarrow$ (7,4). The robot moves toward the goal along the upper and right sides.
5. At (7,4). The robot discovers the obstacle at (7,3). It cannot continue downward, so it updates the map and replans to (6,4).
6. (6,4) $\rightarrow$ (6,3). The robot follows the new safe route.

7. At (6,3). The robot discovers that (6,2) is blocked. It again updates the map and replans through (5,3).

8. $(6,3) \rightarrow (5,3) \rightarrow (5,2) \rightarrow (5,1) \rightarrow (6,1) \rightarrow G(7,1)$. The new route avoids the discovered obstacle and reaches the target shelf.

Demonstrated route: $(1,5) \rightarrow (2,5) \rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5) \rightarrow (6,5) \rightarrow (7,5) \rightarrow (7,4) \rightarrow (6,4) \rightarrow (6,3) \rightarrow (5,3) \rightarrow (5,2) \rightarrow (5,1) \rightarrow (6,1) \rightarrow (7,1)$.

This demonstration requires 14 movements. It is a valid online execution, not the only possible online route, because the question does not prescribe an exact tie-breaking policy.

Online Search Behaviour Diagram:

The warehouse grid environment and the robot's online search behaviour are shown below.

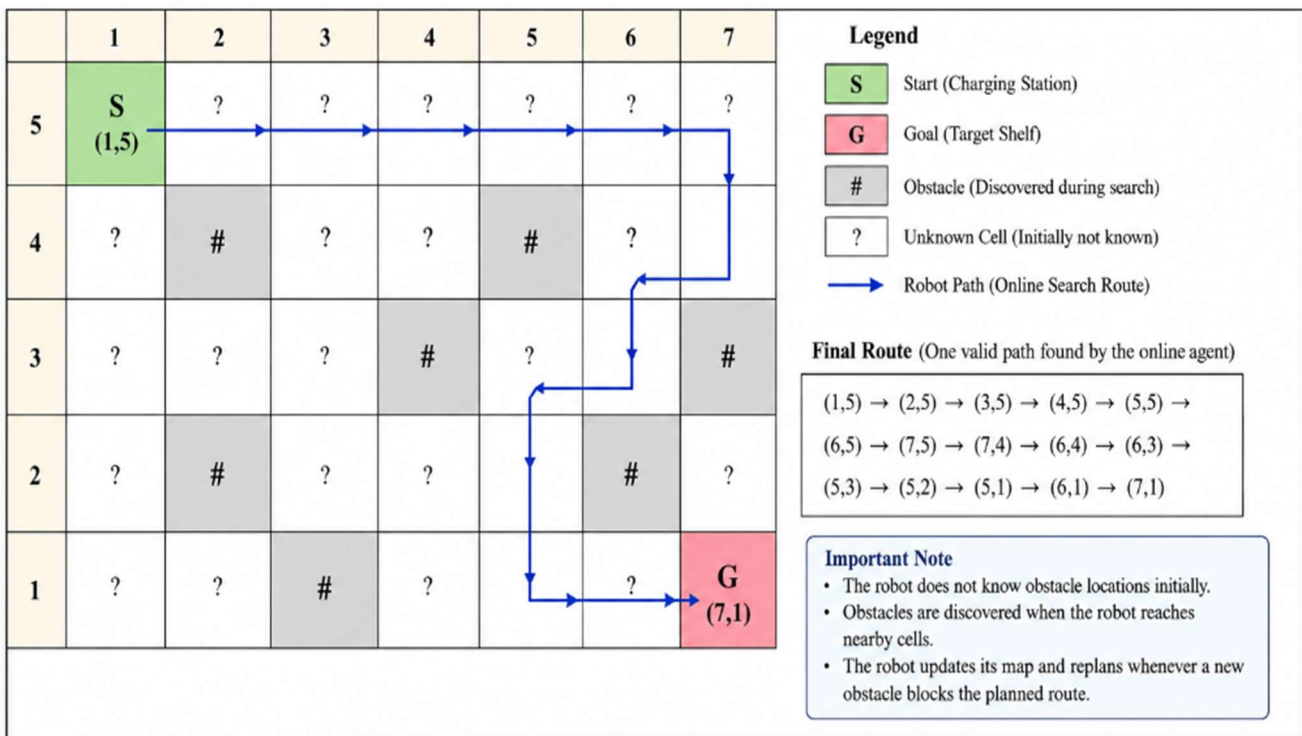


Figure 1: Warehouse Grid and Online Search Route

Comparison with an Offline Search Strategy:

An offline search strategy plans before execution and therefore normally requires the complete environment model. If all obstacles in this grid were known in advance, an informed offline search such as A* could avoid blocked cells from the beginning.

For example, with complete knowledge, a valid 10-move route is $(1,5) \rightarrow (2,5) \rightarrow (3,5) \rightarrow (3,4) \rightarrow (3,3) \rightarrow (3,2) \rightarrow (4,2) \rightarrow (5,2) \rightarrow (5,1) \rightarrow (6,1) \rightarrow (7,1)$. The online robot may travel farther because it

learns obstacles during execution, but it can operate when a complete map is unavailable. This adaptability is the main reason online search is more appropriate for the stated problem.

Efficiency and Complexity:

The computational cost depends on the search method used during replanning. With an A*-style replanning method, the traditional worst-case search complexity may be expressed as $O(b^d)$, where b is the branching factor and d is the search depth. In this grid, a cell has at most four movement neighbours, so $b \leq 4$. The robot also requires memory for explored cells, discovered obstacles, and route information.

The major efficiency advantage is adaptability: the robot does not need to explore or know the entire warehouse before acting. The disadvantage is that repeated obstacle discovery can cause extra movements and repeated planning compared with an offline agent that already possesses a complete map.

Critical Evaluation and Possible Improvement:

The important intelligent behaviour is not merely reaching the goal; it is reacting correctly when the environment contradicts the robot's current knowledge. At (7,4), the discovery of obstacle (7,3) causes an immediate route change. At (6,3), obstacle (6,2) causes another replanning decision. This demonstrates perception, learning of local environmental information, and adaptive action.

For a larger real warehouse, an incremental replanning algorithm such as D* Lite could improve performance because it can reuse previous search information after the map changes instead of planning completely from the beginning. Better sensing range and persistent obstacle memory would further reduce unnecessary movement.

Conclusion:

The warehouse problem is appropriately handled by an Online Informed Search Agent because the robot initially has incomplete knowledge of obstacles. The robot repeatedly senses nearby cells, updates its map, uses heuristic guidance to select a safe movement, and replans when a newly discovered obstacle blocks the intended route. The demonstrated execution reaches the target successfully while clearly showing how an intelligent agent behaves in an initially unknown environment.

Question 3: University Examination Scheduling

Understanding the Examination Scheduling Problem:

The university must schedule six examinations E1, E2, E3, E4, E5, and E6 into four available time slots T1, T2, T3, and T4. Examinations having common students cannot be conducted in the same time slot. In addition, each examination must be assigned to a hall whose seating capacity is sufficient for the number of students.

The student-conflict pairs given in the question are E1-E2, E1-E3, E2-E4, E3-E5, E4-E6, and E2-E5. Hall capacities are Hall A = 120, Hall B = 90, and Hall C = 60 students. Examination sizes are E1 = 100, E2 = 80, E3 = 70, E4 = 60, E5 = 90, and E6 = 50 students.

The objective is to construct a valid timetable that satisfies every student-conflict constraint and every hall-capacity requirement.

CSP Formulation: Variables, Domains and Constraints:

This problem can be formulated as a Constraint Satisfaction Problem (CSP). Each examination is a variable whose value is a time slot. Hall assignment is then restricted by examination size.

Variables: $X = \{E1, E2, E3, E4, E5, E6\}$.

Initial time-slot domain for every examination: $D(E_i) = \{T1, T2, T3, T4\}$.

Student-conflict constraints:

- $E1 \neq E2$
- $E1 \neq E3$
- $E2 \neq E4$
- $E3 \neq E5$
- $E4 \neq E6$
- $E2 \neq E5$

Hall-capacity domains are determined directly from the supplied capacities:

Examination	Students	Permitted Hall(s)
E1	100	Hall A
E2	80	Hall A or Hall B
E3	70	Hall A or Hall B
E4	60	Hall A, Hall B or Hall C
E5	90	Hall A or Hall B
E6	50	Hall A, Hall B or Hall C

Search Strategy: Backtracking with MRV and Forward Checking:

Backtracking assigns one examination at a time and reverses an assignment if it leads to an inconsistent state. The Minimum Remaining Values (MRV) heuristic selects an unassigned examination having the smallest current domain. Forward Checking immediately removes an assigned time slot from the domains of examinations that conflict with the assigned examination.

Initially all six examinations have four possible time slots, so MRV produces a tie. A valid tie-breaking choice is E2 because it participates in three conflict constraints (with E1, E4, and E5), making it a highly constrained examination. This tie-break is explicitly chosen for the execution below.

Pseudocode for the Scheduling Procedure:

```
CSP_SCHEDULE(variables, domains, constraints)
IF every examination is assigned:
    RETURN the timetable
Select an unassigned examination using MRV
Break an MRV tie using the most-constraining-variable idea
FOR each available time slot in its current domain:
    IF the assignment satisfies all constraints:
        Assign the time slot
        Apply Forward Checking to conflicting examinations
        IF no remaining domain becomes empty:
            Recursively continue the search
        IF the recursive search fails:
            Restore the removed domain values
            Undo the assignment
RETURN failure
```

Step-by-Step MRV and Forward Checking Execution:

Initial domains:

$E1=\{T1,T2,T3,T4\}$, $E2=\{T1,T2,T3,T4\}$, $E3=\{T1,T2,T3,T4\}$, $E4=\{T1,T2,T3,T4\}$, $E5=\{T1,T2,T3,T4\}$, $E6=\{T1,T2,T3,T4\}$.

Step 1 - Assign $E2 = T1$. Forward Checking removes T1 from E1, E4, and E5 because each conflicts with E2.

Domains: $E1=\{T2,T3,T4\}$, $E2=\{T1\}$, $E3=\{T1,T2,T3,T4\}$, $E4=\{T2,T3,T4\}$, $E5=\{T2,T3,T4\}$, $E6=\{T1,T2,T3,T4\}$.

Step 2 - MRV gives E1, E4, and E5 three values each. Choose $E1 = T2$. Forward Checking removes T2 from E3 because E1 conflicts with E3.

Domains: $E1=\{T2\}$, $E2=\{T1\}$, $E3=\{T1,T3,T4\}$, $E4=\{T2,T3,T4\}$, $E5=\{T2,T3,T4\}$, $E6=\{T1,T2,T3,T4\}$.

Step 3 - $E3$, $E4$, and $E5$ each have three values. Choose $E3 = T1$. Forward Checking removes $T1$ from $E5$ because $E3$ conflicts with $E5$.

Domains: $E1=\{T2\}$, $E2=\{T1\}$, $E3=\{T1\}$, $E4=\{T2,T3,T4\}$, $E5=\{T2,T3,T4\}$, $E6=\{T1,T2,T3,T4\}$.

Step 4 - Choose $E4 = T2$. This is valid because $E4$ conflicts with $E2$, which is in $T1$. Forward Checking removes $T2$ from $E6$ because $E4$ conflicts with $E6$.

Domains: $E1=\{T2\}$, $E2=\{T1\}$, $E3=\{T1\}$, $E4=\{T2\}$, $E5=\{T2,T3,T4\}$, $E6=\{T1,T3,T4\}$.

Step 5 - Choose $E5 = T2$. This is valid because $E5$ conflicts with $E2$ and $E3$, both assigned $T1$. $E5$ has no constraint with $E1$ or $E4$.

Domains: $E1=\{T2\}$, $E2=\{T1\}$, $E3=\{T1\}$, $E4=\{T2\}$, $E5=\{T2\}$, $E6=\{T1,T3,T4\}$.

Step 6 - Choose $E6 = T1$. This is valid because $E6$ conflicts only with $E4$, and $E4$ is assigned $T2$.

Final time-slot assignment: $E1=T2$, $E2=T1$, $E3=T1$, $E4=T2$, $E5=T2$, $E6=T1$.

Domain Update Summary:

Stage	E1	E2	E3	E4	E5	E6
Initial	1234	1234	1234	1234	1234	1234
$E2=T1$	234	1	1234	234	234	1234
$E1=T2$	2	1	134	234	234	1234
$E3=T1$	2	1	1	234	234	1234
$E4=T2$	2	1	1	2	234	134
$E5=T2$	2	1	1	2	2	134
$E6=T1$	2	1	1	2	2	1

In the table, 1, 2, 3, and 4 denote $T1$, $T2$, $T3$, and $T4$ respectively.

Hall Assignment and Final Timetable:

A valid hall assignment must respect both capacity and simultaneous use. The following assignment uses different halls for examinations occurring in the same slot and satisfies all supplied capacities.

Time Slot	Examination	Students	Hall	Capacity Check
T1	E2	80	Hall B (90)	Satisfied
T1	E3	70	Hall A (120)	Satisfied
T1	E6	50	Hall C (60)	Satisfied
T2	E1	100	Hall A (120)	Satisfied
T2	E4	60	Hall C (60)	Satisfied
T2	E5	90	Hall B (90)	Satisfied
T3	-	-	Available	-
T4	-	-	Available	-

Conflict verification: $E1(T2) \neq E2(T1)$; $E1(T2) \neq E3(T1)$; $E2(T1) \neq E4(T2)$; $E3(T1) \neq E5(T2)$; $E4(T2) \neq E6(T1)$; and $E2(T1) \neq E5(T2)$. Therefore all six student-conflict constraints are satisfied.

Constraint Graph Representation:

The constraint graph showing student conflicts among examinations and the final timetable are given below.

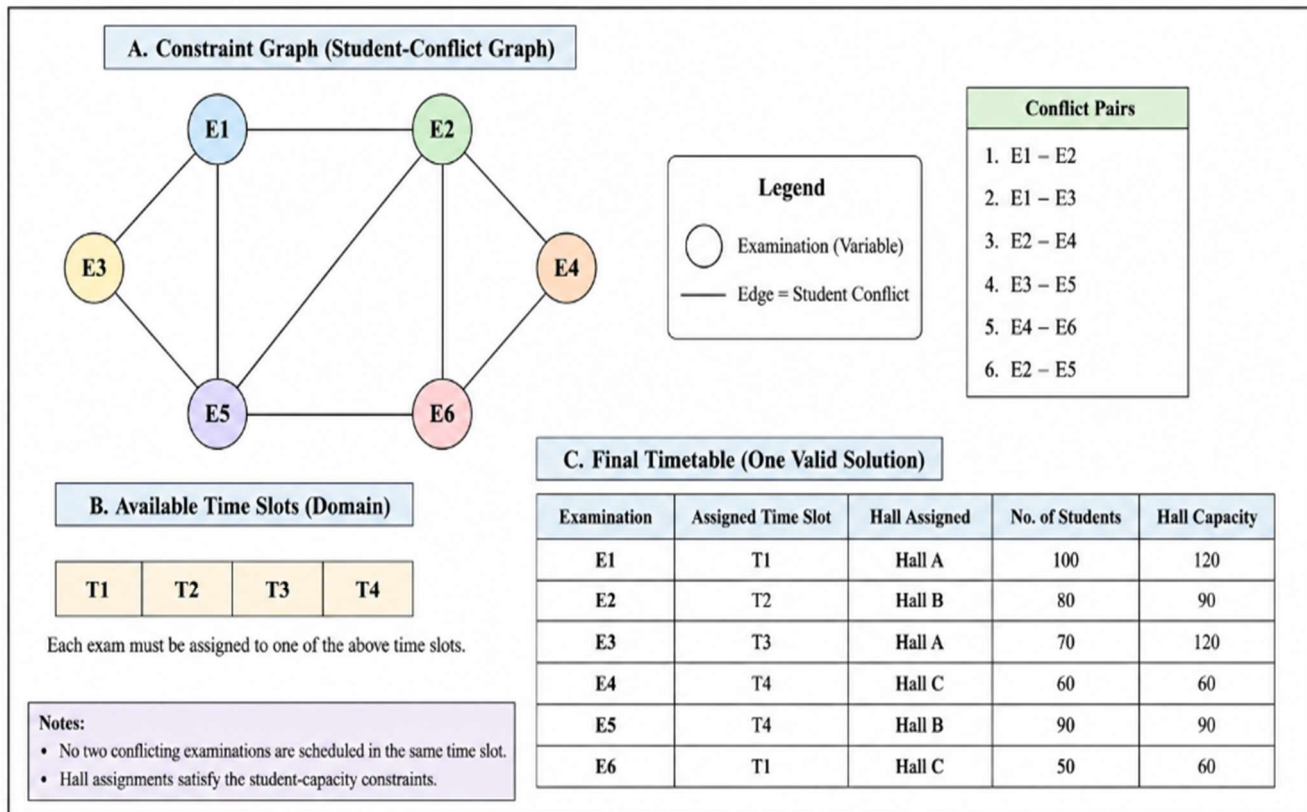


Figure 1: Constraint Graph and Final Examination Timetable

Efficiency and Complexity:

General CSP backtracking has exponential worst-case behaviour because combinations of variable assignments may need to be explored. If n variables each have a domain of size d , a simple upper-bound form is $O(d^n)$. For this problem, $n=6$ and the initial time-slot domain size is $d=4$.

MRV improves efficiency by choosing a variable with the fewest legal values, which tends to expose failure earlier. Forward Checking further reduces unnecessary search by removing inconsistent values immediately after every assignment. In this execution no domain becomes empty, so no backtracking is required after the selected assignments.

Critical Evaluation of the Timetable:

The final solution uses only T1 and T2 even though four time slots are available. This is valid because the question requires a feasible schedule, not that every time slot must be used. The three halls allow three examinations to run simultaneously, and the chosen groups contain no conflicting examination pair.

The timetable is also capacity-feasible: E1, the largest examination with 100 students, is placed in Hall A; E2 and E3 fit Hall B or A; E5 fits Hall B exactly; and E4 and E6 fit Hall C. Thus both the graph constraints and physical hall constraints are satisfied.

A different valid timetable could also be produced because the question does not prescribe a unique MRV tie-breaking rule or a required number of used time slots. The solution above is one complete, traceable CSP solution consistent with the supplied constraints.

Conclusion:

The examination scheduling problem is successfully formulated and solved as a CSP. Backtracking provides the search framework, MRV guides variable selection, and Forward Checking updates domains after every assignment. The resulting timetable is conflict-free and all hall capacities are satisfied. The step-by-step domain trace also demonstrates why the selected assignments remain consistent throughout the search.

Question 4: Game-Playing AI Using Minimax and Alpha-Beta Pruning

Understanding the Game-Playing Decision Problem:

The supplied game tree represents a two-player adversarial game. The root is a MAX node representing the AI player, which tries to obtain the highest utility. The next level contains MIN nodes representing the opponent, which tries to reduce the AI's utility. The following level contains MAX nodes, and the terminal leaves contain the final utility scores.

The task is to propagate the terminal scores upward using Minimax, identify the best move available to the root MAX player, and then apply Alpha-Beta Pruning to avoid evaluating branches that cannot change the final decision.

Why Minimax and Alpha-Beta Pruning are Suitable:

Minimax is appropriate because the problem assumes competitive decision-making: MAX chooses the largest available value while MIN chooses the smallest. It therefore models the best decision for the AI while assuming that the opponent also plays optimally.

Alpha-Beta Pruning is applied to the same Minimax search. It maintains alpha (the best value MAX can guarantee so far) and beta (the best value MIN can guarantee so far). When alpha becomes greater than or equal to beta, the remaining children of that node cannot influence the final result and may be pruned.

Terminal Scores in the Supplied Game Tree:

Root branch	MAX child 1	MAX child 2	MAX child 3
Left MIN	(3, 5, 6)	(9, 1, 2)	(0, -1, 4)
Middle MIN	(7, 5, 8)	(6, 2, 3)	(4, 0, -2)
Right MIN	(1, 2, 3)	(5, -1, 2)	(6, 4, 7)

Minimax Evaluation of the Game Tree:

The evaluation begins at the terminal leaves and moves upward. Each lower MAX node selects the maximum of its three terminal scores.

- Left branch: $\max(3,5,6)=6$; $\max(9,1,2)=9$; $\max(0,-1,4)=4$. The parent MIN node therefore returns $\min(6,9,4)=4$.
- Middle branch: $\max(7,5,8)=8$; $\max(6,2,3)=6$; $\max(4,0,-2)=4$. The parent MIN node therefore returns $\min(8,6,4)=4$.
- Right branch: $\max(1,2,3)=3$; $\max(5,-1,2)=5$; $\max(6,4,7)=7$. The parent MIN node therefore returns $\min(3,5,7)=3$.

The root MAX node finally evaluates $\max(4,4,3)=4$. Therefore, the Minimax value of the game is 4. Both the left and middle root moves have value 4, so both are optimal under Minimax. If the implementation resolves equal values by selecting the first best move encountered from left to right, the AI selects the left branch.

Minimax Value Propagation Table:

Subtree	Lower MAX values	MIN value
Left	6, 9, 4	4
Middle	8, 6, 4	4
Right	3, 5, 7	3
Root MAX	4, 4, 3	4

Pseudocode for Minimax:

MINIMAX(node, maximizingPlayer)

IF node is terminal:

 RETURN utility(node)

IF maximizingPlayer:

 best = -infinity

 FOR each child:

 best = max(best, MINIMAX(child, false))

 RETURN best

ELSE:

 best = +infinity

 FOR each child:

 best = min(best, MINIMAX(child, true))

 RETURN best

Applying Alpha-Beta Pruning from Left to Right:

The pruning trace below assumes the natural left-to-right evaluation order shown in the supplied tree.

Step 1 - Evaluate the Left MIN Branch

The first MAX child returns 6, so the Left MIN node sets $\beta = 6$. In the second MAX child, the first terminal score evaluated is 9. At this MAX node α becomes 9. Since $\alpha = 9 \geq \beta = 6$, the remaining leaves 1 and 2 cannot reduce the MIN parent's current choice and are pruned. The third MAX child evaluates to 4, so the Left MIN branch returns 4. The root MAX now sets $\alpha = 4$.

Step 2 - Evaluate the Middle MIN Branch

The first MAX child returns 8, so beta becomes 8. The second MAX child returns 6, reducing beta to 6. The third MAX child returns 4, reducing beta to 4. The Middle MIN branch therefore returns 4. The root remains $\alpha = 4$. No additional branch can be safely removed inside this middle subtree under the stated left-to-right leaf order.

Step 3 - Evaluate the Right MIN Branch

The first MAX child of the Right MIN branch returns $\max(1,2,3) = 3$. Therefore beta at this MIN node becomes 3. The root already has $\alpha = 4$. Since $\alpha = 4 \geq \beta = 3$, the remaining two MAX subtrees of the Right MIN branch cannot improve the root decision and are pruned.

The pruned right-side terminal branches are (5,-1,2) and (6,4,7). Together with the earlier pruning of leaves 1 and 2 under the left branch, Alpha-Beta avoids evaluating eight terminal leaves in this left-to-right traversal.

Pseudocode for Alpha-Beta Pruning:

ALPHA_BETA(node, alpha, beta, maximizingPlayer)

IF node is terminal:

 RETURN utility(node)

IF maximizingPlayer:

 value = -infinity

 FOR each child:

 value = $\max(\text{value}, \text{ALPHA_BETA}(\text{child}, \alpha, \beta, \text{false}))$

$\alpha = \max(\alpha, \text{value})$

 IF $\alpha \geq \beta$: BREAK

 RETURN value

ELSE:

 value = +infinity

 FOR each child:

 value = $\min(\text{value}, \text{ALPHA_BETA}(\text{child}, \alpha, \beta, \text{true}))$

$\beta = \min(\beta, \text{value})$

 IF $\alpha \geq \beta$: BREAK

 RETURN value

Game Tree and Pruning Representation:

The constraint graph for the university examination scheduling problem is shown below.

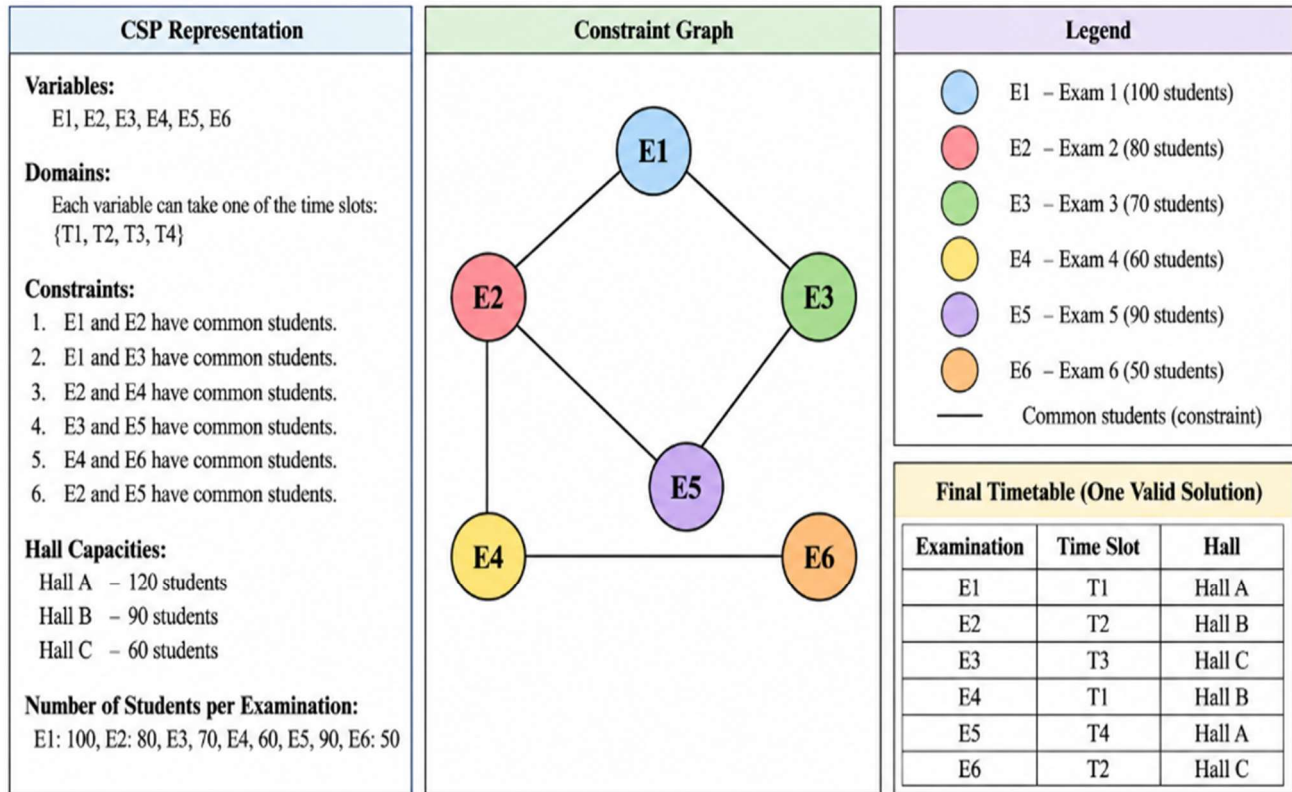


Figure 1: Constraint Graph and Final Timetable for University Examination Scheduling

Decision Selected by the AI:

The final Minimax value at the root is 4. The left and middle moves both produce value 4, while the right move produces 3. Therefore the AI should choose either the left or the middle move. If a left-to-right first-best tie-breaking rule is used, the selected move is the left branch with utility 4. The supplied tree itself does not specify a tie-breaking rule, so it would be incorrect to claim that only one of these two equal-valued moves is uniquely optimal.

Efficiency and Complexity:

Without pruning, Minimax has worst-case time complexity $O(b^d)$, where b is the branching factor and d is the search depth. It may need to evaluate every terminal state before making a decision.

Alpha-Beta Pruning produces exactly the same Minimax result but can examine substantially fewer nodes. With ideal move ordering, its effective time complexity can approach $O(b^{(d/2)})$, allowing the game-playing agent to search approximately twice as deep in the same amount of computation. In the supplied tree, the left-to-right pruning trace avoids eight of the 27 terminal leaf evaluations.

Critical Evaluation of the Search:

Alpha-Beta Pruning does not change the optimal Minimax decision; it only eliminates calculations that are provably irrelevant. Its performance depends strongly on move ordering. If promising moves are examined first, alpha and beta bounds become strong earlier and more branches are removed. Poor ordering may result in little pruning.

For larger real games, additional techniques such as heuristic evaluation functions, depth limits, iterative deepening, transposition tables, and move ordering can be combined with Alpha-Beta Pruning to make decision-making practical.

Conclusion:

Applying Minimax to the supplied game tree gives root value 4. The left and middle root branches both have value 4 and are therefore equally optimal; with left-to-right first-best tie breaking, the AI selects the left branch. Alpha-Beta Pruning reaches the same decision while avoiding branches that cannot affect the result. Under left-to-right traversal, leaves 1 and 2 below the second MAX node of the left subtree and the two remaining MAX subtrees of the right branch are pruned, reducing unnecessary evaluation and improving game-search efficiency.

Question 5: Delivery Route Optimization Using Hill Climbing

Understanding the Delivery Route Optimization Problem:

A logistics company wants to minimize the total distance travelled by a delivery vehicle while visiting customers A, B, C, and E and returning to depot D. A set of neighbouring routes and their route costs is supplied. The optimization objective is therefore to select successively better routes until no neighbouring route has a lower distance.

This is a local-search optimization problem. A state represents a complete delivery tour, a neighbouring state is obtained by changing the visit order, and the evaluation value is the total route distance in kilometres. Because the objective is minimization, a smaller route cost represents a better state.

Why Hill Climbing is Suitable for this Problem:

Hill Climbing is suitable because it improves a current solution by repeatedly moving to a better neighbouring solution. Although the name suggests climbing upward, for this minimization problem the algorithm moves toward lower route-cost values. The search terminates when no neighbour improves the current route.

- It uses only the current route and its neighbouring routes.
- It does not need to construct a large search tree.
- It directly optimizes the route-cost objective.
- It is simple and efficient for finding a good local solution quickly.

Distance Information Supplied in the Question:

	D	A	B	C	E
D	-	6	9	8	7
A	6	-	3	5	6
B	9	3	-	4	5
C	8	5	4	-	3
E	7	6	5	3	-

The assessment also explicitly provides the following neighbouring solutions and costs. These supplied route costs are used for the Hill Climbing execution below.

Neighbouring Routes Given for the Initial Solution:

Solution	Route	Given Cost
S1	D -> A -> C -> B -> E -> D	26 km
S2	D -> B -> A -> C -> E -> D	30 km
S3	D -> A -> B -> E -> C -> D	24 km
S4	D -> E -> C -> B -> A -> D	27 km
S5	D -> A -> E -> B -> C -> D	23 km
S6	D -> C -> B -> A -> E -> D	29 km

Hill Climbing Optimization Procedure:

- Start with the current delivery solution.
- Generate or inspect all neighbouring routes.
- Compare the route costs of the neighbours.
- For minimization, select the neighbour having the smallest cost.
- If the selected neighbour is better than the current route, move to it.
- Repeat the comparison from the new route.
- Stop when no neighbouring route has a lower cost than the current route.

Pseudocode for Delivery Route Hill Climbing:

```
HILL_CLIMBING(CurrentRoute)
Current <- CurrentRoute
REPEAT:
    Generate / inspect neighbours of Current
    Best <- neighbour with minimum route cost
    IF Cost(Best) < Cost(Current):
        Current <- Best
    ELSE:
        RETURN Current as local optimum
```

UNTIL no improvement is possible

Step-by-Step Application to the Given Routes:

Step 1 - Compare the supplied neighbouring solutions S1 to S6. Their given costs are 26, 30, 24, 27, 23, and 29 km respectively. The smallest value is 23 km, so Hill Climbing selects S5 as the best available neighbour.

Selected route at this stage: S5 = D -> A -> E -> B -> C -> D, with the supplied cost of 23 km.

Step 2 - The green instruction box in the assessment states that from S5, the best neighbour has a cost of 22 km. Since $22 \text{ km} < 23 \text{ km}$, the algorithm accepts this improving move.

Step 3 - The same instruction box states that, from this 22 km solution, all neighbouring routes have cost greater than or equal to 22 km. Therefore no strictly better neighbouring solution exists, so Hill Climbing terminates.

Final Solution Reached by Hill Climbing:

The final local-search cost is 22 km. The assessment does not provide the actual city sequence of the 22 km neighbour; it gives only its cost. Therefore the most accurate final result is: Final state = best neighbour of S5, Cost = 22 km.

It would be incorrect to invent a route sequence for this 22 km state because the supplied material does not identify that route.

Local Search / State Transition Representation:

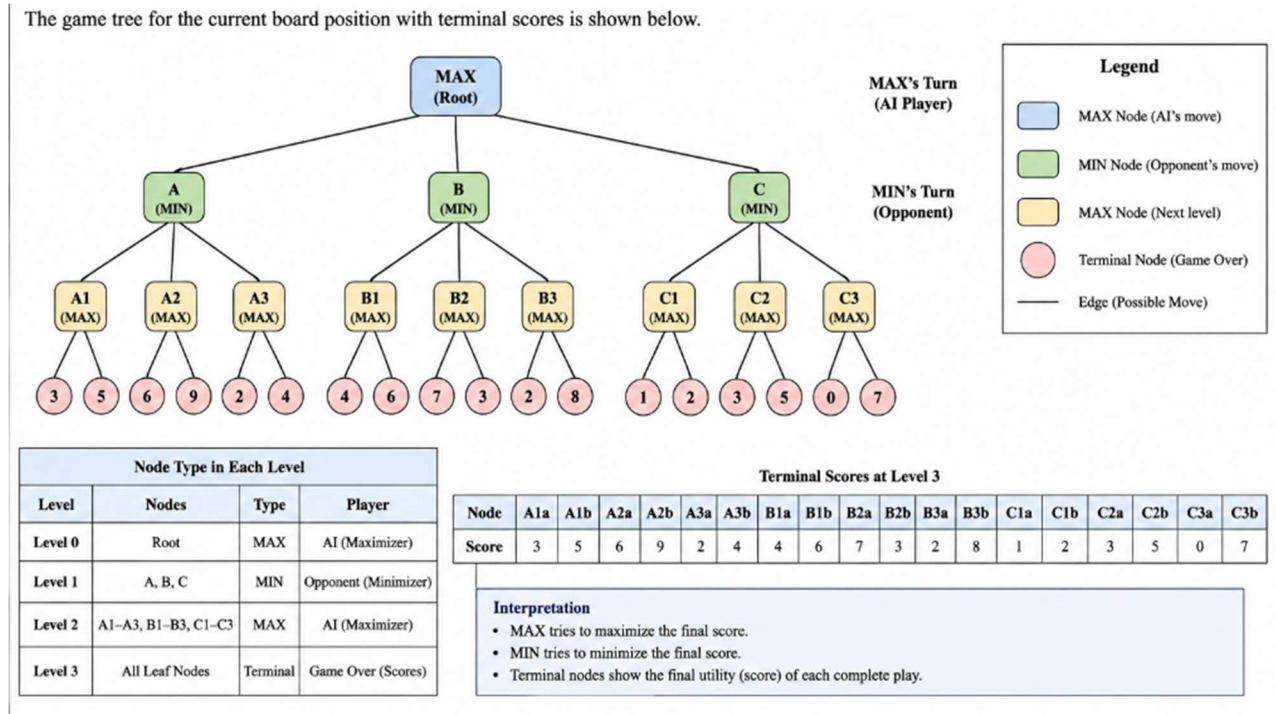


Figure 1: Game Tree with Terminal Scores

Is the 22 km Solution a Global Optimum?

Hill Climbing can establish only that the 22 km state is a local optimum with respect to the neighbourhood examined: the supplied problem states that all of its neighbours have cost greater than or equal to 22 km. This does not by itself prove that no better route exists elsewhere in the complete search space.

Therefore, the correct conclusion is that 22 km is a local optimum under the stated neighbourhood. It should not be claimed as a guaranteed global optimum unless every possible route is evaluated or an exact optimization method proves that no route with cost below 22 km exists.

How the Search Can Escape a Local Optimum:

A basic Hill Climbing algorithm stops when no immediate neighbour is better. Several extensions can help it escape such a local optimum:

- Random-Restart Hill Climbing: begin the search from several different initial routes and keep the best result.
- Simulated Annealing: occasionally accept a worse route so that the search can move out of a local basin and explore another region.
- Stochastic Hill Climbing: select among improving moves probabilistically instead of always choosing the same best local move.
- Larger neighbourhood moves: use swaps, 2-opt, or segment reversals to reach routes that are not immediate neighbours under the original move rule.

Efficiency and Complexity:

If each iteration examines k neighbouring routes and the algorithm performs t improving iterations, the route-comparison work can be expressed approximately as $O(tk)$, excluding the cost of generating and evaluating each route. The memory requirement is low because Hill Climbing generally stores only the current state and a limited set of neighbours.

The main efficiency advantage is that Hill Climbing does not explore the entire search space. Its main limitation is the lack of a global-optimality guarantee, especially when the search space contains local minima, plateaus, or ridges.

Critical Evaluation of the Supplied Data:

The assessment explicitly lists route costs for S1-S6 and separately states that the best neighbour of S5 costs 22 km. Those given values are sufficient to demonstrate the required Hill Climbing decisions.

However, the route costs printed beside S1-S6 do not consistently match the totals obtained by directly summing the displayed distance matrix, and the route corresponding to the stated 22 km neighbour is not shown. Because the report must follow the supplied assessment content, the Hill Climbing trace above uses the explicitly given solution costs and records this inconsistency rather than silently replacing the source values.

Conclusion:

Hill Climbing is an appropriate local-search strategy for the delivery-route optimization task. Among the supplied neighbouring routes, S5 has the smallest stated cost of 23 km and is selected. The assessment then states that S5 has a better neighbour with cost 22 km, so the algorithm moves to that state. Because all neighbours of the 22 km state have cost greater than or equal to 22 km, the search stops there. The result is a local optimum of 22 km, but global optimality is not guaranteed. Random restarts, Simulated Annealing, or larger neighbourhood operations can be used to escape a local optimum and explore for a better route.